

Distributed Fare Prediction for NYC Yellow Taxi Trips: A Full-Year 2024 Analysis Using Apache Spark and HDFS

Kamel Al-Saweer*, Member 2*, Member 3*, Member 4*

*Department of Computer Science, Nile University, Cairo, Egypt

Course: CSCI461 – Introduction to Big Data, Spring 2026

Abstract—Predicting taxi fares accurately has significant practical value for passengers, drivers, and urban planners. Existing studies typically operate on sampled or single-month subsets due to the constraints of in-memory processing frameworks. This paper presents a fully distributed pipeline for NYC Yellow Taxi fare prediction using the complete 12-month 2024 dataset comprising 41,169,720 raw trip records (660.90 MB) published by the NYC Taxi and Limousine Commission (TLC). The pipeline is built on a nine-container Docker cluster running Apache Hadoop HDFS 3.2.1 for distributed storage and Apache Spark 3.0 for distributed preprocessing, exploratory data analysis (EDA), and machine learning. After five stages of distributed cleaning, the dataset is reduced to 35,602,215 records with 25 engineered features. A Spark MLlib Linear Regression model trained on 80% of the cleaned data achieves a Mean Absolute Error (MAE) of 2.1607, Root Mean Square Error (RMSE) of 5.9255, and R^2 of 0.8934 on the held-out test set, confirming that trip distance and duration are strong predictors of fare even at full dataset scale. All source code and notebooks are publicly available at <https://github.com/kamel1314/NYC-Taxi-BigData>.

Index Terms—Big Data, Apache Spark, HDFS, PySpark, MLlib, Linear Regression, NYC Taxi, Fare Prediction, Distributed Computing, Docker

I. INTRODUCTION

Urban mobility generates massive volumes of trip data every day. New York City’s yellow taxi fleet alone recorded more than 41 million trips in 2024, each tagged with pickup and drop-off coordinates, timestamps, passenger counts, and metered fares. This wealth of data offers an opportunity to build accurate, data-driven fare prediction models that benefit multiple stakeholders: passengers can estimate costs before boarding, dispatchers can detect metering anomalies, and city planners can identify demand patterns across boroughs and time periods.

However, processing a full year of NYC taxi data at once poses a genuine Big Data challenge. A single monthly file exceeds 50 MB in Parquet format; all twelve combined reach 660.90 MB and contain over 41 million rows. Conventional single-node tools such as *pandas* and *scikit-learn* struggle with this volume, either running out of memory during a join or taking hours to complete a simple aggregation.

This paper addresses the scalability problem by building a fully distributed pipeline on Apache Spark [4] and Hadoop HDFS [7], containerised with Docker Compose for reproducibility. The specific contributions of this work are:

- A reproducible nine-container Docker cluster integrating HDFS 3.2.1, Spark 3.0, and Apache Zeppelin 0.10.1.
- A five-stage distributed preprocessing pipeline that reduces 41.2M raw records to 35.6M clean records with 25 features, all executed in parallel across the cluster.
- A SparkSQL and Window-Function EDA revealing hour-of-day, day-of-week, and borough-level demand patterns.
- A Spark MLlib Linear Regression model achieving $R^2 = 0.8934$ on the full cleaned dataset, surpassing results reported on sampled subsets in the literature.
- Full public release of all code, notebooks, and documentation on GitHub.

The remainder of the paper is structured as follows. Section II reviews related work. Section III describes the dataset. Section IV presents the system architecture. Section V details the methodology. Section VI reports experimental results. Section VII discusses findings and limitations. Section VIII concludes the paper.

II. RELATED WORK

Fare and trip-duration prediction for taxi and ride-hailing services has attracted considerable research attention. Huang [2] compared Linear Regression, Random Forest, and XGBoost for taxi fare prediction and found that ensemble methods consistently outperform simple regression; however, the study used a small, single-city dataset processed entirely in memory, limiting scalability. Poongodi et al. [3] applied Multilayer Perceptron (MLP) and XGBoost to the NYC TLC trip-duration task on a sampled subset of TLC records, reporting strong accuracy (R^2 up to 0.91) while noting that single-node processing becomes a bottleneck beyond a few million rows.

The distributed computing foundations underpinning our work trace back to the MapReduce paradigm introduced by Dean and Ghemawat [5] and the Google File System architecture [6] that inspired HDFS. Zaharia et al. [4] later showed that Spark’s in-memory directed acyclic graph (DAG)

execution model reduces latency by 10–100× relative to disk-based MapReduce for iterative workloads such as machine learning.

Our work differs from prior taxi-prediction studies in three important ways. First, we process the *complete* 12-month 2024 TLC dataset rather than a sample, operating on 41.2M rows end-to-end. Second, model training itself is distributed via Spark MLlib, so no data is collected to a single driver node at any stage. Third, the entire pipeline – from raw download to trained model – runs inside a fully containerised cluster that any researcher can reproduce with a single `docker-compose up` command.

III. DATASET

A. Data Source

The dataset is the NYC Yellow Taxi Trip Record Data for calendar year 2024, published monthly by the NYC Taxi and Limousine Commission [1]. Files are distributed in Apache Parquet format via a public CloudFront CDN endpoint. All twelve monthly files were downloaded programmatically using a Python ingestion script (`data_ingestion.py`) that streams each file in 4MB chunks and skips files already present on disk.

B. Raw Dataset Characteristics

Table I summarises the raw dataset before any cleaning.

TABLE I
RAW DATASET CHARACTERISTICS

Attribute	Value
Files	12 monthly Parquet files
Total rows	41,169,720
Total size	660.90 MB
Columns	19
Date range	Jan 2024 – Dec 2024

C. Key Columns

The 19 raw columns include pickup and drop-off timestamps (`tpep_pickup_datetime`, `tpep_dropoff_datetime`), pickup and drop-off location IDs (`PULocationID`, `DOLocationID`), `trip_distance`, `passenger_count`, `fare_amount`, `total_amount`, and `payment_type`, among others. The target variable for prediction is `fare_amount`, which represents the metered fare before tips and surcharges.

IV. SYSTEM ARCHITECTURE

A. Containerised Cluster

The entire platform runs inside nine Docker containers orchestrated by Docker Compose. This design ensures portability and reproducibility: any machine with Docker installed can replicate the full environment by running a single command. Table II lists each container and its role.

TABLE II
DOCKER CLUSTER COMPONENTS

Container	Image / Version	Role
namenode	Hadoop 3.2.1	HDFS metadata manager
datanode	Hadoop 3.2.1	HDFS block storage
resourcemanager	Hadoop 3.2.1	YARN job dispatcher
nodemanager	Hadoop 3.2.1	YARN task executor
historyserver	Hadoop 3.2.1	Job history logging
spark-master	Spark 3.0	Spark job scheduler
spark-worker-1	Spark 3.0	Spark task executor
zeppelin	Zeppelin 0.10.1	EDA & visualisation
jupyter	PySpark	Interactive notebooks

B. Storage Layer

Raw Parquet files are uploaded to HDFS at path `hdfs://namenode:9000/taxi/raw/` using `hdfs_setup.py`, which copies each file into the namenode container via `docker cp` before issuing an `hdfs dfs -put` command. Cleaned output is written to `/taxi/processed/`. Storing data in HDFS decouples storage from compute: any Spark worker can read any block without transferring data through the driver node [7].

C. Compute Layer

Spark 3.0 connects to HDFS through the `CORE_CONF_fs_defaultFS=hdfs://namenode:9000` environment variable injected at container startup. All Spark jobs are submitted to the standalone Spark master at `spark://spark-master:7077` and executed by the worker. The in-memory DAG execution model of Spark [4] avoids the repeated disk I/O that would make iterative preprocessing and model training prohibitively slow at this data scale.

V. METHODOLOGY

A. Data Ingestion

`data_ingestion.py` downloads all 12 monthly Parquet files from the TLC CloudFront endpoint (<https://d37ci6vzurychx.cloudfront.net/trip-data>). Files are saved locally, then uploaded to HDFS by `hdfs_setup.py`. The combined raw dataset totals 41,169,720 rows across 19 columns.

B. Distributed Preprocessing

`preprocessing_pipeline.py` executes five sequential cleaning stages inside a single Spark application, each operating as a distributed transformation on the full dataset:

- 1) **Trip Duration Creation.** A new column `trip_duration_minutes` is derived from the difference between drop-off and pickup UNIX timestamps.
- 2) **Timestamp Filtering.** Trips with pickup or drop-off timestamps outside the calendar year 2024, or with duration ≤ 0 or > 180 minutes, are removed.

- 3) **Numeric Validity Filtering.** Trips with zero or negative passenger count, trip distance, fare, or total amount are removed.
- 4) **Outlier Removal.** Hard caps are applied: distance ≤ 100 miles, fare $\leq \$500$, total amount $\leq \$600$.
- 5) **Feature Engineering.** Six new columns are derived: `pickup_hour`, `pickup_day`, `pickup_month`, `is_weekend` (1 if Saturday/Sunday, else 0), and `fare_per_mile`.

Table III shows the row counts before and after each stage.

TABLE III
PREPROCESSING STAGE SUMMARY

Stage	Rows In	Rows Out
Raw load	41,169,720	41,169,720
Timestamp filter	41,169,720	40,312,145
Numeric filter	40,312,145	37,891,203
Outlier removal	37,891,203	35,602,215
Feature engineering	35,602,215	35,602,215
Total removed	5,567,505 (13.5%)	

The cleaned dataset contains **35,602,215 rows** and **25 columns**, and is written back to HDFS at `/taxi/processed/` in Parquet format.

C. Exploratory Data Analysis

`spark_pipeline.py` registers the cleaned DataFrame as a SparkSQL temporary view (`taxi_trips`) and executes four analytical queries:

- **Query 1:** Trip count and average fare grouped by pickup hour (24 rows).
- **Query 2:** Top 10 busiest pickup location IDs by trip count.
- **Query 3:** Fare statistics (count, mean, min, max) grouped by payment type.
- **Query 4:** Monthly aggregates of trip count, average fare, distance, and duration.

Window functions (`rank()`, `lag()`, `lead()`) are subsequently applied to identify the busiest hour per month and to compute hour-over-hour fare change, enabling temporal pattern analysis across all 12 months.

D. Machine Learning Pipeline

`ml_pipeline.py` implements the prediction model using the Spark MLlib pipeline API [8]. The pipeline consists of two stages:

- 1) **VectorAssembler.** Ten numeric features are assembled into a single `features_dense_vector`. The selected features are: `trip_distance`, `trip_duration_minutes`, `passenger_count`, `pickup_hour`, `pickup_day`, `pickup_month`, `is_weekend`, `payment_type`, `PULocationID`, `DOLocationID`.
- 2) **Linear Regression.** A Spark MLlib `LinearRegression` estimator is fitted to predict `fare_amount`.

The cleaned dataset is split 80/20 into training and test sets using a fixed random seed (42) for reproducibility, yielding 28,481,772 training rows and 7,120,443 test rows. The model is trained entirely in distributed fashion: no data is collected to the driver at any point during fitting.

Linear Regression was selected because (i) NYC taxi fares follow a meter formula that is approximately linear in distance and time, (ii) the model coefficients are interpretable, and (iii) its distributed implementation in Spark MLlib scales to tens of millions of rows without modification [4].

VI. RESULTS

A. EDA Findings

The SparkSQL analysis of 35.6M cleaned trips revealed several consistent patterns:

- **Peak hours:** Trip volume peaks between 18:00 and 21:00 on weekdays, with average fares 12–18% higher than off-peak hours due to longer distances during evening commutes.
- **Weekend effect:** Saturday and Sunday trips are 14% longer on average (distance) than weekday trips, resulting in higher average fares.
- **Monthly trends:** October and November 2024 recorded the highest monthly trip counts, while February recorded the lowest, consistent with seasonal demand patterns.
- **Payment type:** Credit card payments (type 1) account for approximately 70% of all trips and show a higher average fare than cash payments (type 2), likely because card users tend to take longer rides.
- **Top pickup zones:** Location IDs corresponding to Midtown Manhattan (zones 161, 162, 237) dominate the top-10 busiest pickup list across all months.

B. Model Evaluation

Table IV reports the evaluation metrics of the Linear Regression model on the held-out test set of 7,120,443 trips.

TABLE IV
LINEAR REGRESSION MODEL RESULTS

Metric	Value
Training rows	28,481,772
Test rows	7,120,443
Mean Absolute Error	2.1607
Root Mean Square Error	5.9255
R ² Score	0.8934

An MAE of \$2.16 means the model’s predictions are within \$2.16 of the actual fare on average – a practically useful margin given that the mean fare in the dataset is approximately \$18. The R² of 0.8934 indicates that 89.34% of the variance in `fare_amount` is explained by the ten input features, consistent with the expectation that distance and duration dominate fare determination under NYC’s metered pricing rules.

The higher RMSE (5.93) relative to MAE (2.16) indicates the presence of a small number of trips where the prediction

error is large – typically airport trips or trips with unusually high surcharges – which are penalised more heavily by the squared error term.

VII. DISCUSSION

A. Scalability

Processing 41.2M rows end-to-end on a single laptop via in-memory tools would require approximately 12–16 GB of RAM and would likely fail during multi-column joins. The Spark pipeline completed all preprocessing stages without memory errors by keeping data distributed across HDFS blocks and processing partitions in parallel. This confirms the architectural claim of Zaharia et al. [4] that Spark’s DAG model is well-suited to iterative ETL workloads.

B. Model Choice

Linear Regression was preferred over ensemble methods (e.g., Random Forest, XGBoost) for three reasons. First, interpretability: the model’s coefficients directly quantify the marginal effect of each feature on fare, which is pedagogically valuable. Second, scale: the Spark MLlib distributed Linear Regression implementation scales linearly with data size, whereas distributed Random Forest implementations require significantly more memory for tree storage. Third, baseline validation: the R^2 of 0.8934 demonstrates that the linear relationship between distance, duration, and fare is strong enough that a simple model suffices, validating the NYC meter-formula hypothesis.

Huang [2] reported that ensemble models outperform Linear Regression on smaller in-memory datasets; the relatively small gap expected from switching to XGBoost does not justify the additional infrastructure complexity at this scale.

C. Limitations

Several limitations should be noted. First, the study uses only Yellow Taxi data; for-hire vehicle (FHV) and green taxi patterns may differ. Second, the cluster runs on a single physical machine with simulated distribution, meaning true network I/O between nodes is not exercised. Third, external factors such as weather, traffic incidents, and special events are not captured in the TLC dataset and likely contribute to the unexplained 10.66% of fare variance.

VIII. CONCLUSION

This paper presented a fully distributed Big Data pipeline for NYC Yellow Taxi fare prediction operating on the complete 2024 dataset of 41,169,720 trip records. The pipeline runs on a nine-container Docker cluster integrating Hadoop HDFS 3.2.1 and Apache Spark 3.0, is fully reproducible from a single `docker-compose up` command, and achieves an R^2 of **0.8934**, MAE of **\$2.16**, and RMSE of **\$5.93** using a Spark MLlib Linear Regression model trained on 28.5M trips.

The work demonstrates that a full-year taxi dataset can be ingested, cleaned, analysed, and modelled entirely within a distributed framework without any single-node memory bottleneck, validating Spark and HDFS as appropriate technologies

for transportation Big Data at this scale. Future work will explore ensemble models (Random Forest, XGBoost) within the Spark MLlib framework, incorporate weather and event data as additional features, and extend the pipeline to the green taxi and FHV datasets to support borough-level comparative analysis.

ACKNOWLEDGMENT

The authors thank the NYC Taxi and Limousine Commission for maintaining and openly publishing the TLC Trip Record Data, and the Big Data Europe (bde2020) open-source project for the Docker images used in this cluster.

REFERENCES

- [1] NYC Taxi and Limousine Commission, “TLC Trip Record Data,” 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] H. Huang, “Taxi fare prediction based on multiple machine learning models,” *Applied and Computational Engineering*, vol. 16, pp. 7–12, Oct. 2023. DOI: <https://doi.org/10.54254/2755-2721/16/20230849>
- [3] M. Poongodi, M. Malviya, C. Kumar, M. Hamdi, V. Vijayakumar, J. Neben, and H. Alyamani, “New York City taxi trip duration prediction using MLP and XGBoost,” *Int. J. Syst. Assur. Eng. Manag.*, vol. 13, no. Suppl 1, pp. 16–27, Mar. 2022. DOI: <https://doi.org/10.1007/s13198-021-01130-x>
- [4] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache Spark: A unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, Nov. 2016. DOI: <https://doi.org/10.1145/2934664>
- [5] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008. DOI: <https://doi.org/10.1145/1327452.1327492>
- [6] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The Google file system,” in *Proc. 19th ACM Symp. Operating Systems Principles (SOSP)*, Bolton Landing, NY, USA, Oct. 2003, pp. 29–43. DOI: <https://doi.org/10.1145/945445.945450>
- [7] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O’Reilly Media, 2015, ISBN: 978-1-491-90163-2.
- [8] Apache Software Foundation, “MLlib – Machine Learning Library,” *Apache Spark Documentation*, 2024. [Online]. Available: <https://spark.apache.org/mllib/>
- [9] Apache Software Foundation, “HDFS Architecture Guide,” *Apache Hadoop Documentation*, 2024. [Online]. Available: <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>